CSCI160 Computer Science I
Lab Exercise: File Processing, Histograms and counting (two approaches)

This lab introduces two approaches to processing and counting data from an external text file.

Objectives:

- Follow instructions carefully, focusing on the current module's topics.
- Read data from a file and process its contents.
- Utilize loops and conditionals to extract and analyze data
- Implement lists and indexing for character frequency counting. Data must be kept in logical order: A, B, C, D . . . Z
- Implement a dictionary to count data where order is not important
- Write clean, readable, and well-documented Python code.
- Display frequency data using a bar graph format.
- Cite code accessed through AI or Internet resources.

NOTE: If you use any code or techniques that are not included in our current lesson or suggested by AI, tutor, or any Internet source, you must cite and thoroughly explain them in comments—even if you wrote the code from scratch.

Failure to do so will result in an automatic 20% point deduction. No exceptions. If you are unsure, reach out for clarification.

Lab Problem #1: Letter Frequency Histogram → *histogram.py*

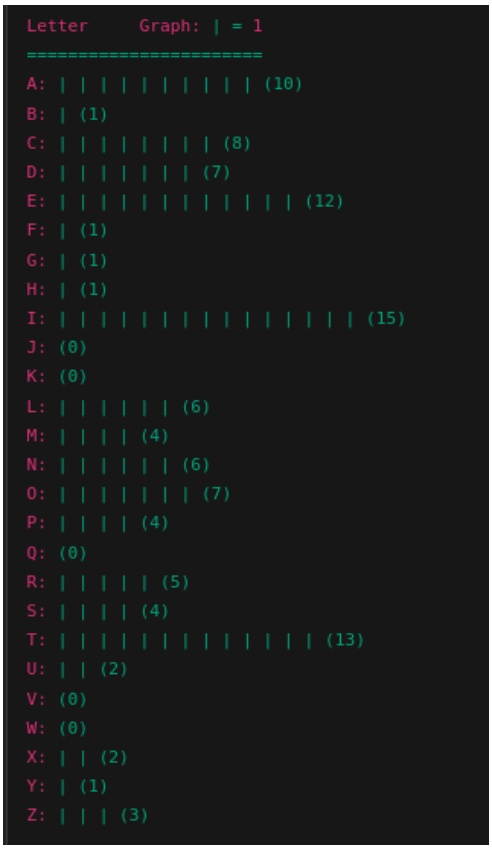
Problem Description:

- You have been provided with a collection of sample text files
- Read a text file and generate a histogram displaying the frequency of each letter (A-Z).
- Convert all letters to uppercase for uniformity.
- Ignore non-alphabetic characters.
- Use a list with 26 elements, where each index corresponds to a letter (A = 0, B = 1, ..., Z = 25).
- Use the `ord()` function to map letters to their respective indices.
- Display the histogram as a bar graph using `|` symbols, where each `|` represents one occurrence of a letter.

Sample Run (File Contents):

```
Assimilated
orchestration
Compatible
optimizing
Recontextualized
Decentralized
didactic
complexity
functionalities
adapter
```

Would generate this histogram



A histogram is a plot that lets you discover, and show, the underlying frequency distribution (shape) of a set of continuous data. This allows the inspection of the data for its underlying distribution (e.g., normal distribution), outliers, skewness, etc.

Instead of explicitly counting characters with 26 branches like so . . .

```
if   letter == 'A' then a-count += 1
elif letter == 'B' then b-count += 1
```

We are going to count characters by placing them into `buckets`. For our purposes a bucket is going to be a position in a list. The fundamental concept here is to convert our bucket input (English letters) to list indices. If we can immediately convert the input to its associated list index we can avoid writing 26 decision statements.

Fundamental to this is the conversion of the string input data to numeric data. This concept is called a `hash`. Luckily for us, characters can easily be converted back and forth to numbers via the `ord` and `chr` functions. We will just be worrying about capital letters so we will need a list with 26 buckets . . . one for each character in our alphabet. Realize that a list of 26 elements will have an index range of 0 - 25

Steps:

- Create a list of size 26: **`histogram = [0] * 26`**
  - A's will be counted in bucket **`histogram[0]`**
  - B's will be counted in bucket **`histogram[1]`**
  - C's will be counted in bucket **`histogram[2]`**
  - . . . etc . . .

if the histogram list is structured like this: **`[11, 2, 7, 4]`**

```
histogram[ 0 ] = 11 # this means there are 11 A's
histogram[ 1 ] = 2  # this means there are 2  B's
histogram[ 2 ] = 7  # this means there are 7  C's
histogram[ 3 ] = 4  # this means there are 4  D's
```

How can we tell Python that As are counted in bucket 0? How can we map an A to a zero?

- Take it's ordinal and subtract 65.
- **ord('A') --> 65 . . . (65 – 65) --> 0**
- **ord('B') --> 66 . . . (66 – 65) --> 1**
- **ord('C') --> 67 . . . (67 – 65) --> 2**
- Why 65? The ordinal of 'A' is 65 and we A's to be counted at position 0
- Use the hash results as an **index into the histogram list and add one to that location**. You will end up with a list of numbers where each number is the count of that specific letter in the variable.

Display the results to the terminal as a bar graph. The number of bars "|" to print for each letter will be based on the quantity at each list location

```
histogram[0] = 3
histogram[1] = 2
histogram[2] = 4
```

would render the bar graph. Review the image on page 1 to see the complete histogram for one sample file.

```
A: | | | (3)
B: | | (2)
C: | | | | (4)
```

Implementation Steps:

1. **Read the input file**
    - Ensure that the text files are in the same directory as your Python source code file. Also make sure that you have the correct folder open in VS Code.
    - Open the file and read its contents line by line.
        - Use Python's `open` function to open the file
        - Use one of the read functions to bring the data from the file into your program's memory. Realize that there are multiple approaches to solving this. I recommend either
            - Using a `for loop` to iterate line by line

```
try:
    input_file = open("input.txt", "r") # open the file in read mode
    for line in input_file:             # loop through each line in the file
        print(line)                     # process each line
        # do a bunch of cool stuff with the line
        # like having a nested for loop to count the characters
    input_file.close()                  # after the loop, close the file
except FileNotFoundError as fnf:        # This code runs if the file is not found
    print(f"File not found: {fnf}")     # print the error message
```

- Using the `readlines` function to place the file data into a list. One line per list position

```
try:
    input_file = open("input.txt", "r")  # open the file in read mode
    lines = input_file.readlines()       # read all lines in the file into a list
    for line in lines:                   # loop through each line in the file
        print(line)                      # process each line
        # do a bunch of cool stuff with the line
        # like having a nested for loop to count the characters
    input_file.close()                   # after the loop, close the file
except FileNotFoundError as fnf:         # This code runs if the file is not found
    print(f"File not found: {fnf}")      # print the error message
```

- Strip newline characters and spaces. Use Python's `strip` function for this

2. **Count letter occurrences using a list**
    - Create a list `histogram` of size 26 initialized with zeros.
    - Loop through each character in the file:
        - Convert it to uppercase.
        - If it is a letter (`A-Z`), determine its index using `ord(letter) - 65`.
        - Increment the corresponding index in the `histogram` list.
3. **Display the histogram as a bar graph**
    - Print the header.
    - Loop through the `histogram` list and display:
        - The letter.
        - A series of `|` characters representing the letter's count.
        - The numerical count in parentheses.

Lab Problem #2: Determining the winner of an election → *election.py*

In this problem, you will write a Python program to analyze election data and determine the winning party in the US Presidential election based on county-level results. We also want to be able to

- Show the totals by state
- Show the state with the widest margin of victory
- Show the state with the tightest margin of victory

Problem Breakdown

The data is provided in a CSV file containing county-level election results per state. This file contains (C)`omma (S)eparated (V)alues` but can be read simply as a text file containing lines of data. You are not allowed to use any Python libraries (like the `csv or pandas` module) that do this work for you. Stick with this week's techniques and concepts.

!! – I make no claims on the accuracy of this data – !!

The file follows this structure where each row represents the election results for a specific county.

`state_name,county_fips,county_name,votes_gop,votes_dem,total_votes,diff,per_gop,per_dem,per_point_diff`

Here are the first few lines of the file. I have opened the file in VS Code. If you double click the file to open it, it will probably open it in Excel or some sort of spreadsheet. **Don't do that!**

2024_US_County_Level_Presidential_Results.csv > data

```
state_name,county_fips,county_name,votes_gop,votes_dem,total_votes,diff,per_gop,per_dem,per_point_diff
Alabama,01001,Autauga County,20484,7439,28190,13045.0,0.7266406527137282,0.2638879035118836,0.4627527492018446
Alabama,01003,Baldwin County,95798,24934,121808,70864.0,0.7864672271115197,0.20469919873899908,0.5817680283725206
Alabama,01005,Barbour County,5606,4158,9832,1448.0,0.5701790073230268,0.4229048006509357,0.14727420667209112
Alabama,01007,Bibb County,7572,1619,9241,5953.0,0.8193918407098799,0.1751974894491938,0.6441943512606861
Alabama,01009,Blount County,25354,2576,28115,22778.0,0.9017961942023831,0.09162368842255024,0.8101725057798329
Alabama,01011,Bullock County,1101,2983,4106,-1882.0,0.2681441792498782,0.7264978080857282,-0.45835362883584996
Alabama,01013,Butler County,5172,3251,8467,1921.0,0.6108420928309909,0.3839612613676627,0.22688083146332821
Alabama,01015,Calhoun County,34912,13194,48530,21718.0,0.7193900679991758,0.2718730682052339,0.44751699979394194
Alabama,01017,Chambers County,8711,5405,14221,3306.0,0.6125448280711624,0.38007172491385977,0.23247310315730263
Alabama,01019,Cherokee County,11358,1553,12984,9805.0,0.8747689463955638,0.11960874922982132,0.7551601971657425
Alabama,01021,Chilton County,16920,2698,19728,14222.0,0.8576642335766423,0.13675993511759935,0.720904298459043
Alabama,01023,Choctaw County,4103,2515,6646,1588.0,0.6173638278663858,0.3784231116461029,0.23894071622028284
```

We are not trying to read spreadsheet files in this problem. Do not approach it that way. It is a simple text file and we'll be using Python's `open` function and reading lines just like the previous problem. If you want the pretty colors in your VS Code display, install the `Rainbow CSV extension`.

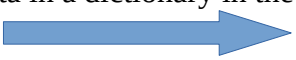
Notice how each line begins with the state name. You will also find the county name and fields for total GOP votes and total DEM votes. Those are the fields we are interested in.

GOAL: Accumulate sum totals for all `votes_gop` and `votes_dem` and `organize them by state`

The key to structuring our memory to efficiently process this data is to use a `dictionary.` Similar to the histogram problem we do not want to do something like this

```
if    state == "Alabama" then increment_alabama_votes
elfif state == "Alaska"  then increment_alaska_votes
elfif state == "Arizona" then increment_arizona_votes
```

This type of code is ugly and difficult to debug and maintain. We want to structure our data in a dictionary in the following way



```
votes_by_state = {
    # state   : [votes_gop, votes_dem]
    "Alabama" : [1500,      1250],
    "Alaska"  : [1000,      500],
    "Arizona" : [2000,      2500],
}
```

Now, before you start typing out all 50 state names . . . STOP, because it is not necessary!

We are going to start with an `empty dictionary` to which we'll be adding states that aren't already present OR adding to to states that are present.

```
# empty dictionary to hold the votes by state
votes_by_state = {}
```

Dictionary Sum Logic

- Get the state and the `votes_gop/votes_dem`
- Using the membership operator (`in`) check to see if the state is already in the dictionary
  - If it IS in the dictionary, accumulate the new `votes_gop/votes_dem` to the existing
  - If it IS NOT in the dictionary, add the state as a key and insert the `votes_gop/votes_dem`

Example using sample data

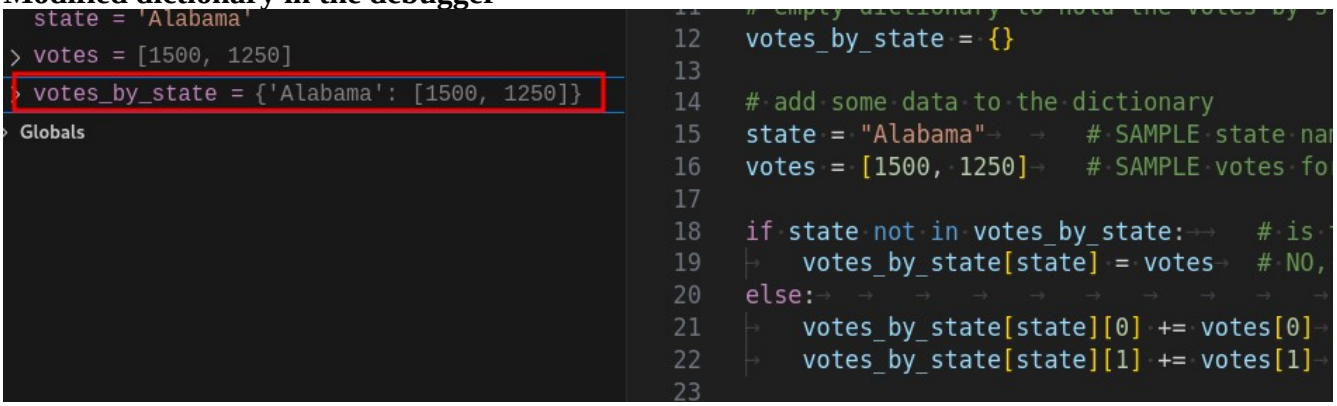
```
# empty dictionary to hold the votes by state
votes_by_state = {}

# add some data to the dictionary
state = "Alabama"      # SAMPLE state name
votes = [1500, 1250]   # SAMPLE votes for GOP and Dem

if state not in votes_by_state:    # is the state already in the dictionary
    votes_by_state[state] = votes  # NO, add the state and votes to the dictionary
else:                              # YES, the state is already in the dictionary
    votes_by_state[state][0] += votes[0]   # add the GOP votes to the existing state
    votes_by_state[state][1] += votes[1]   # add the DEM votes to the existing state
```

Notice how the dictionary is no longer empty by watching in the debugger. When we get new totals for a different county in Alabama, the state won't be added but the totals will be increased.

Modified dictionary in the debugger



Hints on data reading

2024_US_County_Level_Presidential_Results.csv > data

```
state_name,county_fips,county_name,votes_gop,votes_dem,total_votes,diff,per_gop,per_dem,per_point_diff
Alabama,01001,Autauga County,20484,7439,28190,13045.0,0.7266406527137282,0.2638879035118836,0.4627527492018446
Alabama,01003,Baldwin County,95798,24934,121808,70864.0,0.7864672271115197,0.20469919873899908,0.5817680283725206
Alabama,01005,Barbour County,5606,4158,9832,1448.0,0.5701790073230268,0.4229048006509357,0.14727420667209112
Alabama,01007,Bibb County,7572,1619,9241,5953.0,0.8193918407098799,0.1751974894491938,0.6441943512606861
Alabama,01009,Blount County,25354,2576,28115,22778.0,0.9017961942023831,0.09162368842255024,0.8101725057798329
Alabama,01011,Bullock County,1101,2983,4106,-1882.0,0.2681441792498782,0.7264978080857282,-0.45835362883584996
Alabama,01013,Butler County,5172,3251,8467,1921.0,0.6108420928309909,0.3839612613676627,0.22688083146332821
Alabama,01015,Calhoun County,34912,13194,48530,21718.0,0.7193900679991758,0.2718730682052339,0.44751699979394194
Alabama,01017,Chambers County,8711,5405,14221,3306.0,0.6125448280711624,0.38007172491385977,0.23247310315730263
Alabama,01019,Cherokee County,11358,1553,12984,9805.0,0.8747689463955638,0.11960874922982132,0.7551601971657425
Alabama,01021,Chilton County,16920,2698,19728,14222.0,0.8576642335766423,0.13675993511759935,0.720904298459043
Alabama,01023,Choctaw County,4103,2515,6646,1588.0,0.6173638278663858,0.3784231116461029,0.23894071622028284
```

1. Notice that there are multiple lines `for each state`
2. Make sure the CSV file is in the same directory as your source code file
3. Use Python's `open` function to open the file
4. Choose how you want to read the data
    1. All at once into a list using `readlines()`
    2. Or one line at a time inside a loop using `readline()`
    3. Both work just fine, but are slightly different in the details
5. Be sure to remove the newline character and any other junk using `strip()`
6. The easiest way to deal with a single line is to split it into a list of tokens using the comma as the split point

Split example using sample line from file

```
line = "Alabama,01019,Cherokee County,11358,1553,12984,9805.0,0.8747689463955638,0.11960874922982132,0.7551601971657425"
# split the line at commas into a list of values
line = line.split(",")
# line[0] = state name
# line[1] = county code
# line[2] = county name
```

Debugger View of line list, post split

```
line = ['Alabama', '01019', 'Cherokee Cou…
> special variables
> function variables
  0 = 'Alabama'
  1 = '01019'
  2 = 'Cherokee County'
  3 = '11358'
  4 = '1553'
  5 = '12984'
  6 = '9805.0'
  7 = '0.8747689463955638'
  8 = '0.11960874922982132'
  9 = '0.7551601971657425'
  len() = 10
```

Now you can easily get the state name and vote totals by using their position in the list. Refer to the file header to determine which position contains the GOP/DEM votes.

Given the data in this file, the results are

```
Total votes: 152301538
Total GOP votes: 77294799
Total DEM votes: 75006739
GOP percentage: 50.75%
DEM percentage: 49.25%
GOP wins!
```

Steps to Process Votes by State:

1. **Initialize an empty Dictionary**
2. **Read and Process the File**:
    - Be sure to skip the header row!!
    - Extract **state name**, **GOP votes**, **DEM votes** from each line.
    - Update the dictionary by inserting or accumulating votes for each state.
3. **Display Results by iterating through the dictionary and computing**
    - Total votes
    - Total GOP votes
    - Total DEM votes
    - Percentage for each party
    - The winning party
4. **Extra Challenges**
    - Show the state with the least total votes
    - Show the state with the most total votes
    - Show the state with the widest margin.
    - Show the state with the tightest margin.
    - Compute and display the percentage of votes won by each party in every state
    - Allow the user to enter a state name and show the totals only for that state
    - Show the states whose voting majority was for GOP
    - Show the states whose voting majority was for DEM

Submission: Attach your two Python files for this assignment (histogram.py and election.py). Do not submit data files. I have my own that I'll be using.